

Root-Mask-and-Skeletons — System Architecture

Executive Summary

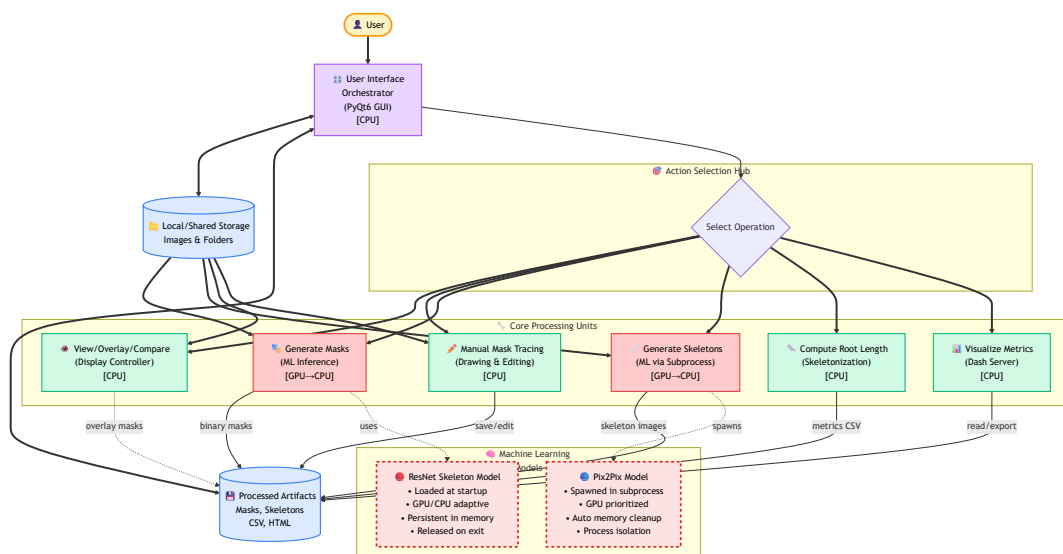
The Root-Mask-and-Skeletons application is a comprehensive PyQt6-based system for automated root analysis in plant science. It combines computer vision, deep learning, and interactive visualization to process root images, generate segmentation masks, extract skeletal structures, and compute morphological metrics.

Key Capabilities

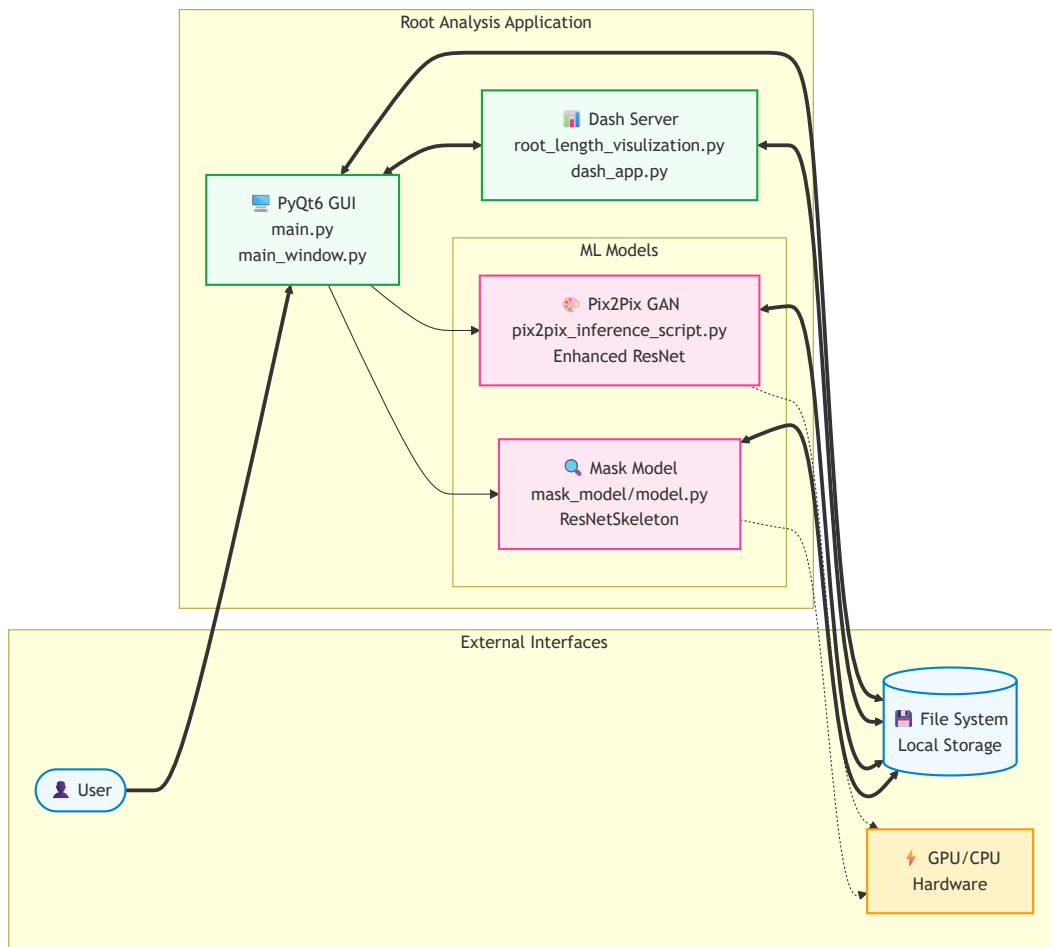
-  **Interactive Mask Editing:** Manual tracing and editing tools for precise segmentation
-  **ML-Powered Processing:** Dual-model architecture using ResNet for masks and Pix2Pix for skeletons
-  **Advanced Analytics:** Automatic root length calculation and morphometric analysis
-  **Rich Visualizations:** Interactive dashboards with Plotly Dash integration
-  **Hardware Optimization:** Intelligent GPU/CPU resource management

System Architecture Overview

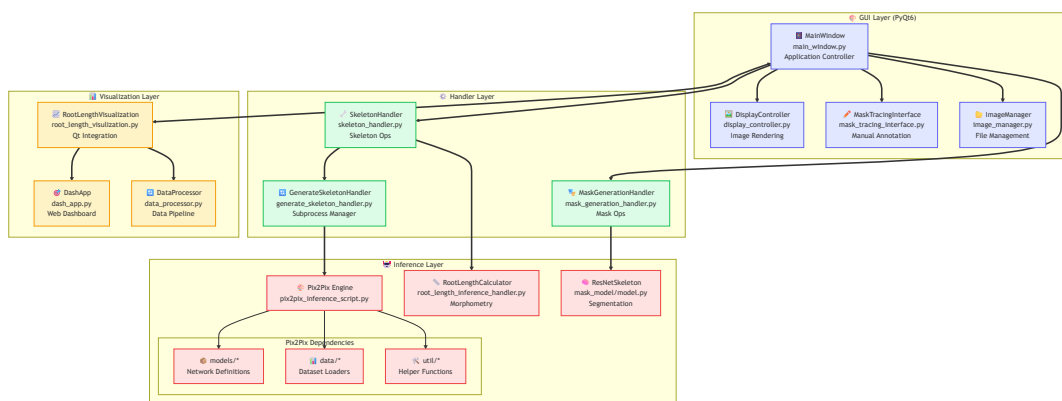
Unified Functional Architecture



System Context & External Interfaces

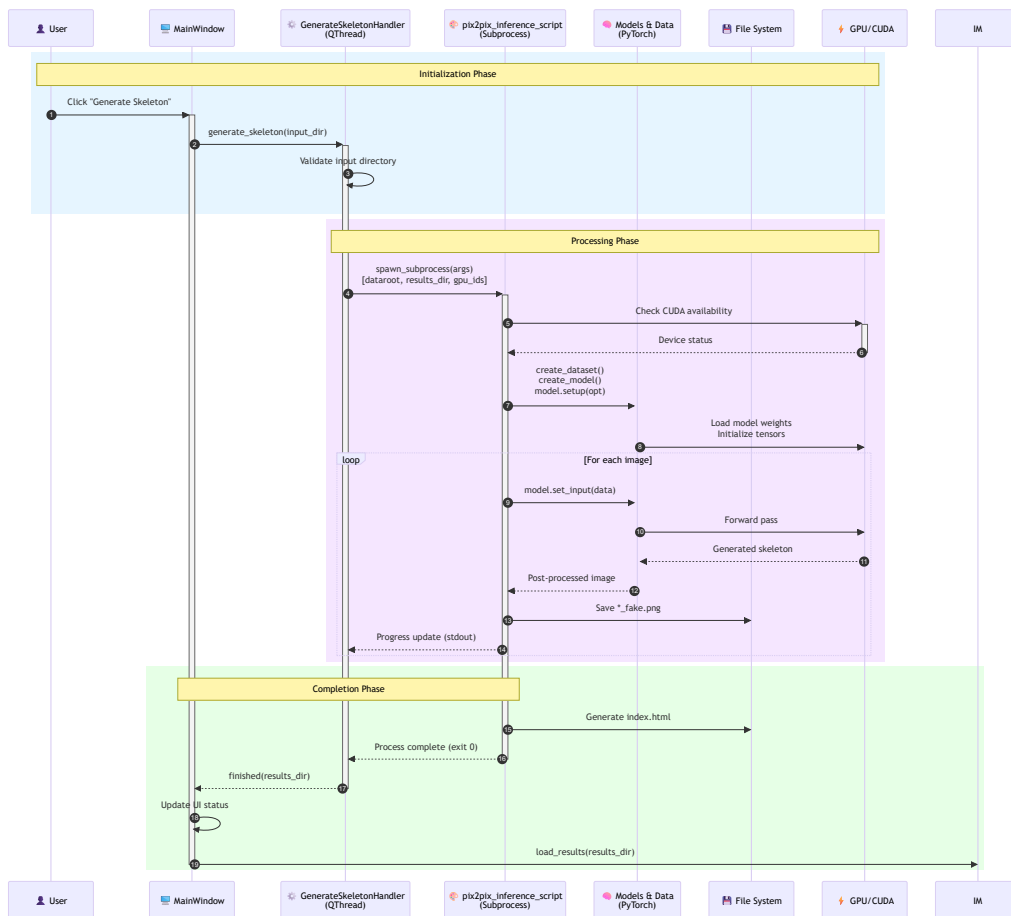


🔌 Component Architecture & Module Dependencies

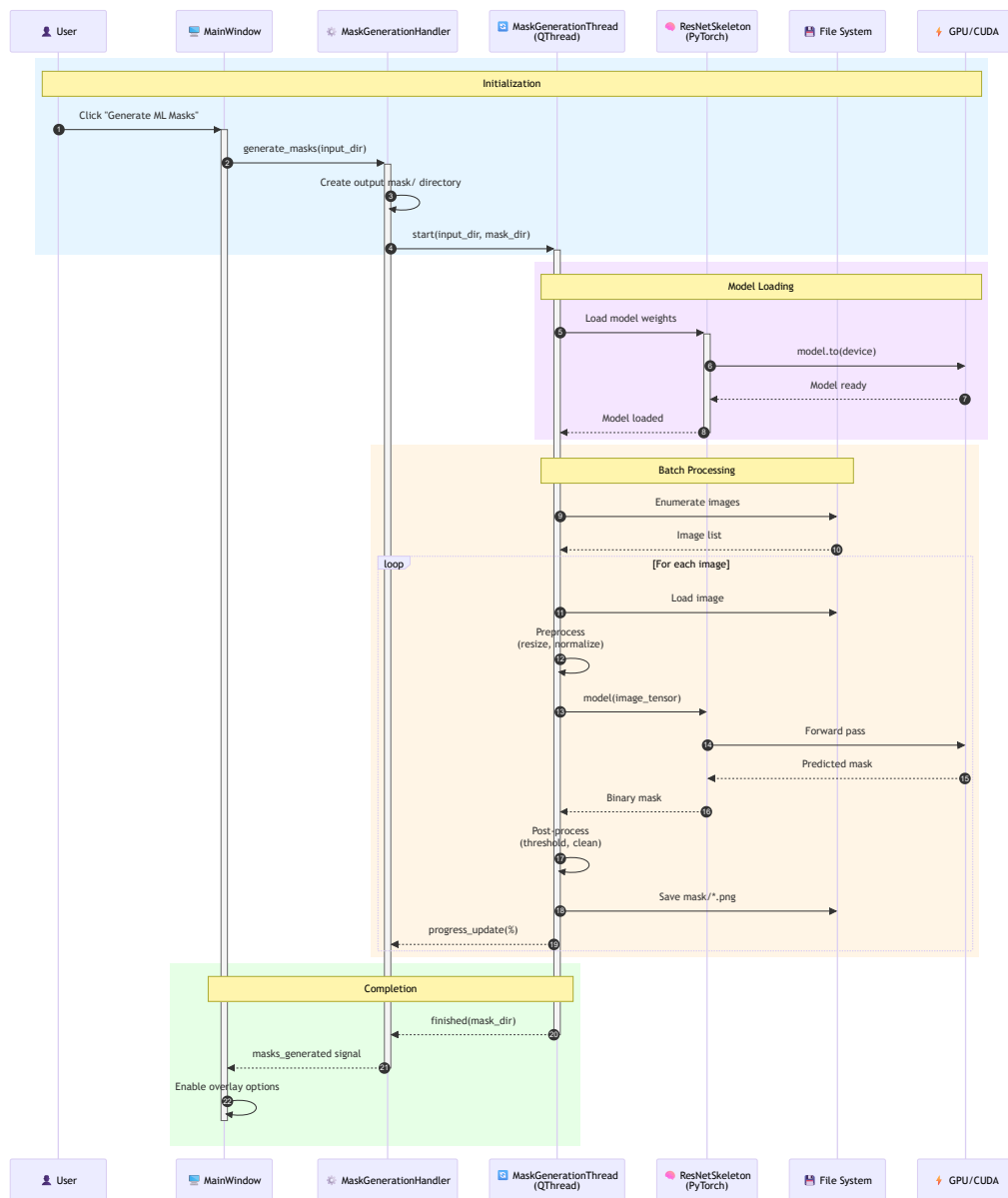


🔄 Operational Workflows

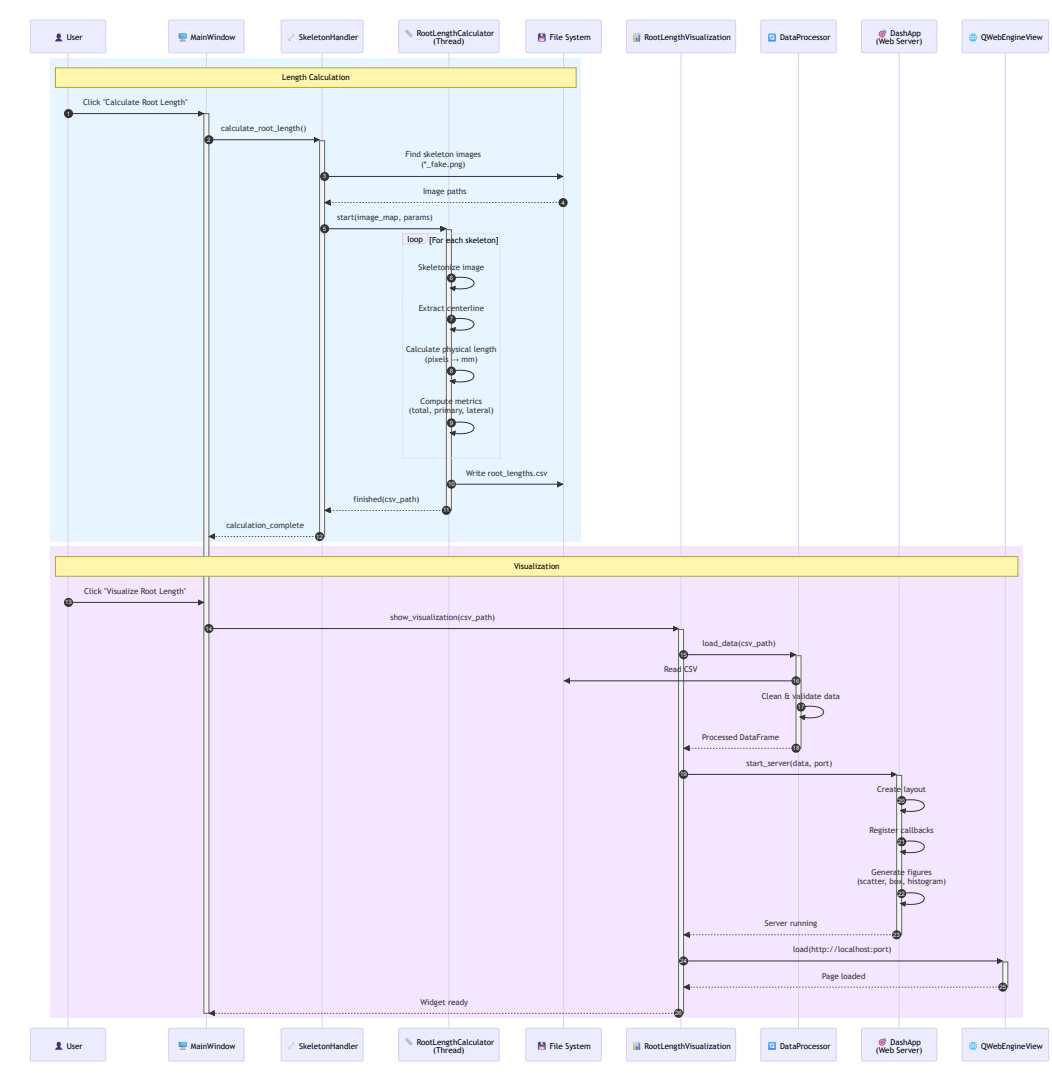
🔗 Skeleton Generation Workflow (Pix2Pix GAN)



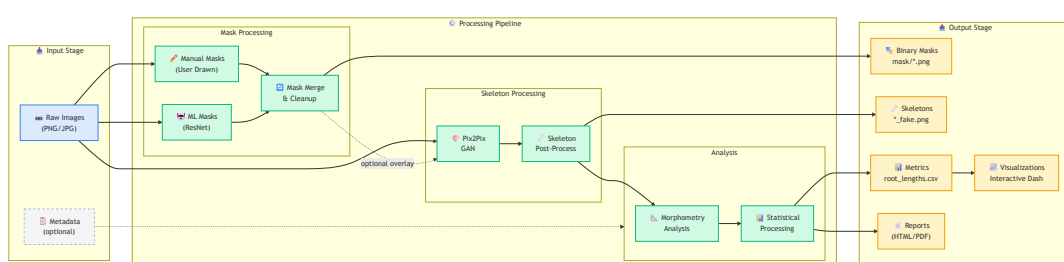
Mask Generation Workflow (ResNet CNN)



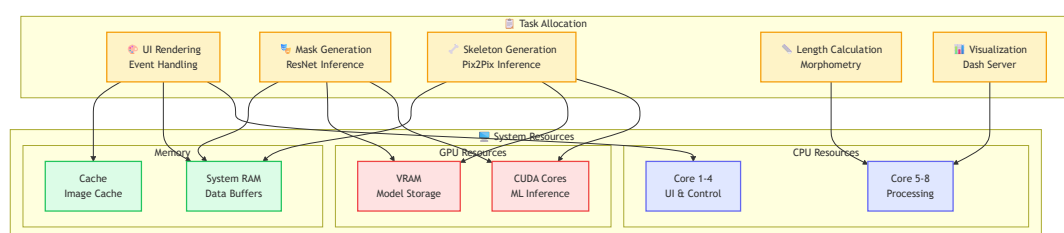
Root Length Calculation & Visualization



Data Flow Architecture

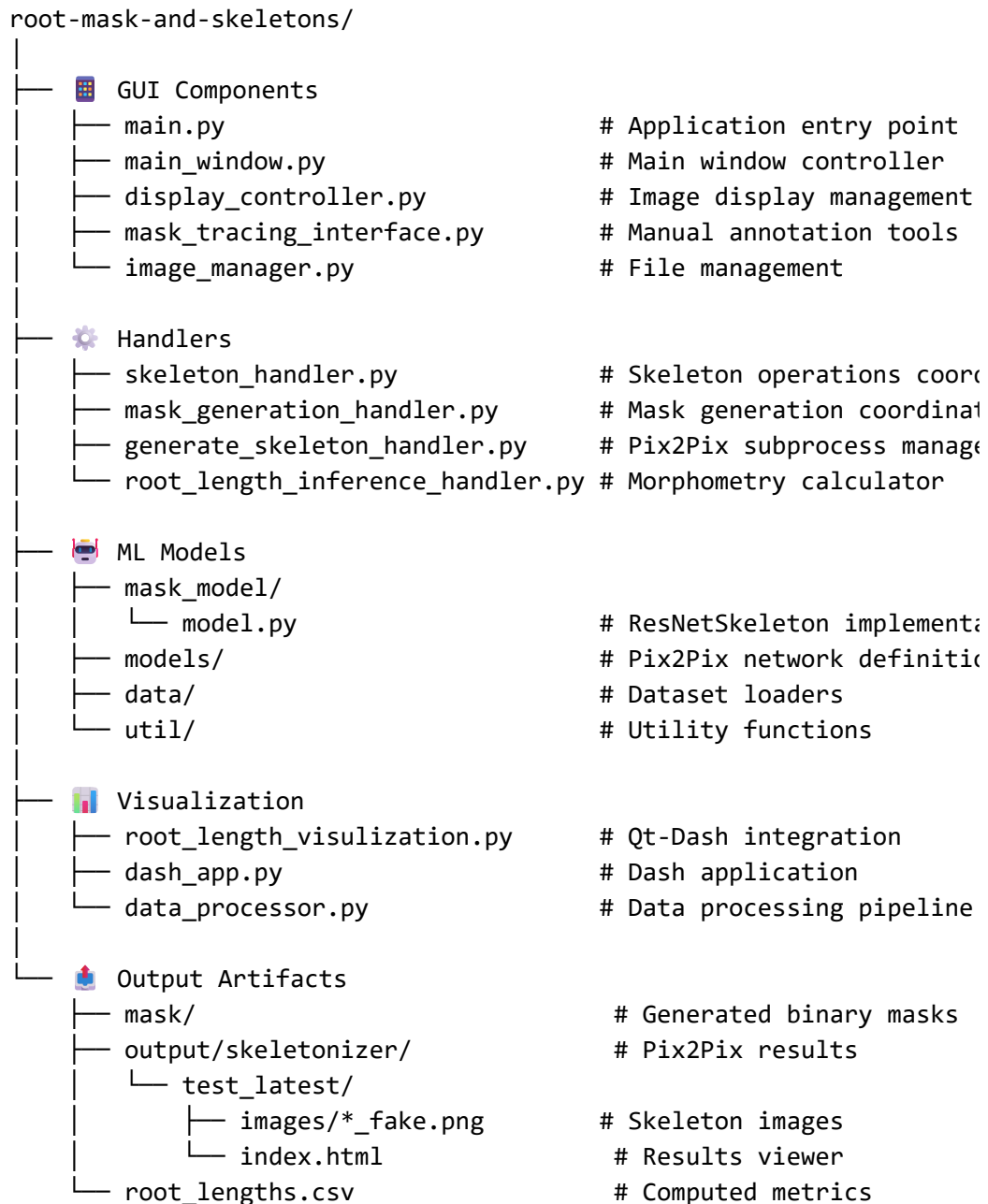


Hardware Resource Management



File Structure & Artifacts

Directory Structure



Runtime Artifacts

Artifact Type	Location	Format	Description
Input Images	User specified	PNG/JPG	Original root images
Binary Masks	mask/*.png	8-bit PNG	Segmentation masks
Skeleton Images	*_fake.png	RGB PNG	Skeletonized roots

Metrics CSV	root_lengths.csv	CSV	Morphometric data
HTML Reports	index.html	HTML	Visual results browser
Dash Server	localhost:8050	Web	Interactive dashboard

Recommended Requirements

- **CPU:** Intel i7/i9 or AMD Ryzen 7/9 (8+ cores)
- **RAM:** 16-32 GB
- **GPU:** NVIDIA RTX 3060 or better (8+ GB VRAM)
- **Storage:** 50 GB SSD

 Performance Optimization Strategies

GPU Utilization

- **Model Persistence:** ResNet model loaded once and kept in memory
- **Batch Processing:** Process multiple images in single GPU transfer
- **Mixed Precision:** FP16 computation where supported
- **Memory Management:** Automatic VRAM cleanup after subprocess completion

CPU Optimization

- **Multithreading:** Separate threads for UI, processing, and visualization
- **Async Operations:** Non-blocking file I/O and network requests
- **Cache Strategy:** LRU cache for frequently accessed images
- **Lazy Loading:** Load data only when needed

Memory Management

- **Image Pyramids:** Multi-resolution representation for large images
- **Streaming Processing:** Process large datasets in chunks
- **Garbage Collection:** Explicit cleanup after major operations
- **Resource Pooling:** Reuse objects and buffers where possible